

Phase1:Installation&Setup

1.Install Java&Verify

Code:

```
sudo apt update
```

```
sudo apt install openjdk-21-jdk-y
```

```
echo 'export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64' >> ~/.bashrc
source ~/.bashrc
```

```
java -version
```

```
cd /tmp
```

```
wget https://dlcdn.apache.org/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
```

```
tar xzf spark-4.1.1-bin-hadoop3.tgz
```

```
sudo mv spark-4.1.1-bin-hadoop3/opt/spark
```

```
echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc
echo 'export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin' >> ~/.bashrc
echo 'export PYSPARK_PYTHON=/usr/bin/python3' >> ~/.bashrc
```

```
source ~/.bashrc
```

```
echo $SPARK_HOME
```

Phase2: Your First Spark Exercise

```
pyspark
```

```
data = spark.sparkContext.parallelize([ "Hello
world Hello Spark",
"Spark is great for big data", "Hello
Python and Spark"
])
```

```
df = data.toDF(schema=["line"])
```

```
from pyspark.sql.functions import explode, split, col, desc
```

```
words_df = df.select(explode(split(col("line"), "")).alias("word"))
```

```
sorted_word_counts.show()
```

Output:

```

sky@MS97377:/tmp$ sudo apt update
[sudo] password for sky:
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.5 kB]
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:3 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3680 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [650 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3411 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [6341 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [482 kB]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [6105 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [1215 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1279 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [1172 kB]
Get:15 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1046 kB]
Get:16 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [234
kB]
Fetched 25.9 MB in 36s (723 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
6 packages can be upgraded. Run 'apt list --upgradable' to see them.
sky@MS97377:~$ sudo apt install openjdk-21-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-21-jdk is already the newest version (21.0.11+10-1~22.04.2).
0 upgraded, 0 newly installed, 0 to remove and 6 not upgraded.
sky@MS97377:~$ java -version
openjdk version "21.0.11" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11+10-1~22.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.11+10-1~22.04.2-Ubuntu, mixed mode, sharing)
sky@MS97377:~$ echo 'export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64' >> ~/.bashrc
sky@MS97377:~$ source ~/.bashrc
sky@MS97377:~$ echo $JAVA_HOME
/usr/lib/jvm/java-21-openjdk-amd64
sky@MS97377:~$ cd /tmp
sky@MS97377:/tmp$ pwd
/tmp
sky@MS97377:/tmp$ wget https://dlcdn.apache.org/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
2026-07-28 08:32:11 - https://dlcdn.apache.org/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:de42:644
sky@MS97377:/tmp$ wget https://archive.apache.org/dist/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
- 2026-07-28 08:33:06-- https://archive.apache.org/dist/spark/spark-4.1.1/spark-4.1.1-bin-hadoop3.tgz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:a1:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 572739279 (546M) [application/x-gzip]
Saving to: 'spark-4.1.1-bin-hadoop3.tgz'

spark-4.1.1-bin-hado 100%[=====] 546.21M  955KB/s   in 9m 27s

2026-07-28 08:42:34 (987 KB/s) - 'spark-4.1.1-bin-hadoop3.tgz' saved [572739279/572739279]

sky@MS97377:/tmp$ ls -lh spark-4.1.1-bin-hadoop3.tgz
-rw-r--r-- 1 sky sky 547M Jan  2  2026 spark-4.1.1-bin-hadoop3.tgz
sky@MS97377:/tmp$ tar xzf spark-4.1.1-bin-hadoop3.tgz
sky@MS97377:/tmp$ ls
sparkdata-sky
spark-private-tmp
spark-4.1.1-bin-hadoop3
spark-4.1.1-bin-hadoop3.tgz
systemd-private-7c8be3d42b5444dfb349e33056d7157e-systemd-logind.service-eYdzfA
systemd-private-7c8be3d42b5444dfb349e33056d7157e-systemd-resolved.service-1mSUXG
systemd-private-7c8be3d42b5444dfb349e33056d7157e-systemd-timesyncd.service-6z7Hp7
sky@MS97377:/tmp$ sudo mv spark-4.1.1-bin-hadoop3 /opt/spark
sky@MS97377:/tmp$ ls /opt
containerd  spark
sky@MS97377:/tmp$ echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc
sky@MS97377:/tmp$ echo 'export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin' >> ~/.bashrc
sky@MS97377:/tmp$ echo 'export PYSARK_PYTHON=/usr/bin/python3' >> ~/.bashrc
sky@MS97377:/tmp$ source ~/.bashrc
sky@MS97377:/tmp$ echo $SPARK_HOME
/opt/spark
sky@MS97377:/tmp$ spark-submit --version
WARN: Using incubator modules: jdk.incubator.vector
Using spark's default 16j profile: org/apache/spark/log4j2-defaults.properties
26/07/28 08:44:33 WARN Utils: Your hostname, MS97377, resolves to a loopback address: 127.0.1.1; using 172.29.63.103 instead (on interface eth0)
26/07/28 08:44:33 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Welcome to

```


Reflection Questions:

1. DataFrames vs. RDDs: Why might a DataFrame be more efficient than an RDD for this word count task? (Hint: Think about Spark's Catalyst Optimizer.)

A **DataFrame** is generally more efficient than an **RDD** because it uses **Spark's Catalyst Optimizer**, which automatically analyzes and optimizes the execution plan. It can optimize operations such as filtering, grouping, and aggregation, reducing execution time and resource usage. DataFrames also use Spark's optimized execution engine, making them faster and more memory-efficient than RDDs for structured data processing.

2. Lazy Evaluation: In the PySpark shell, when does the actual computation for `word_counts` happen? Is it when `groupBy` and `count` are defined, or when `show()` is called?

Spark follows the concept of **Lazy Evaluation**. Methods such as `groupBy()` and `count()` are **transformations**, so they only build an execution plan and do not perform the computation immediately. The actual computation starts only when an **action** such as `show()` is called. At that point, Spark executes all the pending transformations and produces the final result.

3. Partitions: How could the `spark.sparkContext.parallelize()` function be used to control the parallelism of the job?

The `parallelize()` function accepts an optional **second argument** that specifies the number of **partitions**. More partitions allow Spark to process data in parallel across multiple CPU cores, which can improve performance for larger datasets. Choosing an appropriate number of partitions helps balance the workload and use system resources efficiently.